Question 1: Banking Domain – Enforcing Account Invariant

Scenario:

You are designing a `BankAccount` class.

Business Rules:

- Every account must have:
  - accountNumber (cannot be changed once created)
  - accountHolderName
  - balance
- Balance can never be negative.
- If balance is not provided during account creation, it should default to ₹0.
- Interest rate is same for all accounts and should be initialized only once.
- A PremiumAccount extends BankAccount.
- PremiumAccount must add bonus amount during account creation.

Tasks:

1. Design constructors for:
   - BankAccount
   - PremiumAccount
2. Ensure that:
   - accountNumber is immutable
   - balance never becomes negative during object creation
3. Use:
   - constructor overloading
   - `this()` for reusability
   - `super()` to initialize parent properly
4. Explain:
   - Why constructor must enforce "balance ≥ 0"
   - Why we should not allow object creation without valid state
5. What would break if someone directly sets balance after object creation?

Question 2: Insurance Claim System – Controlled Initialization & State Flow

Scenario:

Design a `InsuranceClaim` class.

Each claim has:

- claimId (must be auto-generated)
- policyNumber
- claimAmount
- status (Initially must be "Filed")
- approvedAmount (initially 0)

Rules:

- claimAmount cannot be negative.
- Claim must always start in "Filed" state.
- A `CorporateClaim` extends InsuranceClaim.
- CorporateClaim must set a processing fee during construction.
- claimId must NOT be passed manually from outside.
- All objects must be created in a valid state only.

Tasks:

1. Decide which constructor parameters are mandatory.
2. Decide which constructor should be private (if any).
3. Use:
    - Parameterized constructor
    - Constructor chaining
    - `super()` in inheritance
4. Ensure:
    - Invalid object cannot be created.
5. Explain:
    - Why constructor is better place for validation than setters.
    - Why child constructor must call parent constructor first.
    - What happens if super() is not called (conceptually)?

Question 3: E-Commerce Product & Order System – Object Dependency & Creation Control

Scenario:

You are designing:

- Product
- Order
- DiscountedOrder (extends Order)

Rules:

Product:

- productId (immutable)
- price (must be > 0)
- stockQuantity
- Cannot create product without price.

Order:

- orderId
- Product reference
- quantity
- Total amount must be calculated at creation time.
- Order cannot exist without a Product.
- Quantity must be > 0.

DiscountedOrder:

- Accepts discount percentage during construction.
- DiscountedOrder must calculate discounted total.
- Parent constructor must calculate base total first.

Tasks:

1. Design constructors for:
    - Product

    - Order
    - DiscountedOrder
2. Use:
    - `this()` where reusable
    - `super()` for proper parent initialization
3. Prevent invalid object creation like:
    - Negative price
    - Null product
    - Zero quantity
4. Explain:
    - Why order should calculate total inside constructor.
    - Why parent initialization should not be skipped.
    - Why constructor is better than setter in this scenario.
5. What could go wrong if you compute total later instead of during construction?